Non-relational database management system

Design specification

Lukas Stavaris, Emilis Juknevičius

November 2026

Contents

1 Introduction **2**
- 1.1 Overview . . . 2
- 1.2 The problem we are solving . . . 2
- 1.3 How DBMS differs from existing tools . . . 2
- 1.4 Core audience . . . 3
- 1.5 Value proposition . . . 3
- 1.6 Non-goals . . . 3

2 Definitions **3**

3 Features and business requirements **4**
- 3.1 Must have . . . 4
- 3.2 Should have . . . 6
- 3.3 Could have . . . 6
- 3.4 Will not have . . . 6

4 Business flows **6**
- 4.1 Inserting a record . . . 6
- 4.2 Querying with an index . . . 7
- 4.3 Deleting records and compacting a page . . . 8
- 4.4 Startup and crash recovery . . . 9
- 4.5 Creating a secondary index . . . 10

5 Non-functional requirements **10**
- 5.1 Performance . . . 10
- 5.2 Durability and consistency . . . 11
- 5.3 Concurrency . . . 11
- 5.4 Capacity, limits and resource usage . . . 11
- 5.5 Reliability, observability and security . . . 12

6 High-level architecture **12**
- 6.1 Package structure . . . 12
- 6.2 Class structure . . . 13
- 6.3 Mapping requirements onto packages . . . 14
- 6.4 Logical model . . . 15
- 6.5 Keys and how values are compared . . . 15
- 6.6 Physical model . . . 16
- 6.7 Byte layouts . . . 17
- 6.8 How indexes find records . . . 19
- 6.9 Writing safely: the log, checkpoints and free space . . . 20
- 6.10 The catalog . . . 21

7 Interface: HTTP API and query language **21**
7.1 Query language 21
7.2 Result paging 22
7.3 Endpoints 22
7.4 Status codes and error codes 23

1 Introduction

1.1 Overview

This document specifies the functionality of a small non-relational database management system, later refered to as DBMS. DBMS stores schemaless JSON records, keeps an entire database in a single file on disk, and is driven exclusively over an HTTP interface. It is designed to be implementable by a small team in one development cycle: the whole system is a paged file, a hash index, a query parser and a HTTP server.

A DBMS database is a single file of fixed-size *pages*, with records packed into them in a flexible layout, so records of very different sizes sit side by side without padding or a declared schema. Every record belongs to a *collection* and carries a unique *primary field*; looking a record up by that field costs a single page read, because the key is hashed to find the page directly rather than searching. Any other field can be given a *secondary index* to make it equally fast to query. There is no shell, no desktop client and no administration console: the database is reached only through its HTTP API, which carries a deliberately small query language. The server itself is started from a command line and stopped with a signal.

1.2 The problem we are solving

Developers who need to store structured data locally face an uncomfortable gap. On one side is the *ad-hoc file*: data kept in hand-written JSON, loaded wholly into memory and rewritten on every change. It is free to start with and works until the file outgrows memory, two processes need to write to it, or an interrupted write truncates it. There is no indexing, so every lookup is a full scan, which is very slow.

On the other side is the *database server*. PostgreSQL or MongoDB solves all of that, but the cost is operational rather than technical: a service to install, configure, keep running, secure, back up and upgrade. For a desktop application, a game, a prototype or a device at the edge of a network, that machinery outweighs the problem. Relational servers additionally demand a schema up front, which is precisely what a project still changing shape cannot provide.

DBMS targets the gap between the two. It gives a project real indexing, real crash durability and a real query interface, while remaining a single file and a single process with nothing to administer. The schemaless record model lets fields be added as the application evolves, without migrations.

1.3 How DBMS differs from existing tools

Embedded and single-file stores already exist, so the niche has to be stated precisely.

- **SQLite** (with its JSON functions) is the closest neighbour: one file, a write-ahead log, no server. But it is a relational engine reached by linking a library or installing a driver, and "schemaless" means JSON text inside a column queried through functions. DBMS's record model is schemaless natively and its interface is language-neutral HTTP.
- **LiteDB and similar embedded document stores** are single-file and schemaless but tied to one language runtime.
- **LMDB and RocksDB** are embedded key-value stores: fast, but with no query language and no secondary indexes out of the box.
- **MongoDB and PostgreSQL** offer everything DBMS does and far more, at the price of running a server, which requires knowledge to set up.

DBMS's position is the combination none of these offers at once: a single data file, schemaless JSON documents, equality queries served by hash indexes on any field, crash safety, and an HTTP interface usable from any language without a driver.

1.4 Core audience

- **Small teams and solo developers** whose application has outgrown a JSON file but does not justify operating a database server.
- **Desktop and game developers** needing local schemaless persistence – save data, player profiles, item catalogues – that survives a crash and ships inside the application rather than as a dependency the user must install.
- **Prototype and internal-tool authors** who want to store documents immediately, query them by any field, and not design a schema first.
- **Edge and embedded services** on constrained hardware, where one modest process with a bounded memory budget is the only acceptable footprint.

1.5 Value proposition

- **Nothing to administer.** Create a database by naming a file – no users, roles or configuration files – and back it up or ship it with a file copy.
- **Fast lookups by construction.** Hash indexing makes an equality lookup a constant-time operation on the primary field and on any indexed field.
- **Schemaless.** Records in a collection need not share a shape, and new fields require no migration.
- **Crash-safe.** A write-ahead log means an interrupted write leaves the database consistent, not truncated.
- **Reachable from anything.** The interface is HTTP and JSON, so any language can use it without a driver.

1.6 Non-goals

DBMS is not a relational database: no joins, no foreign keys, no SQL. It is not a distributed system – one database, one process, one machine, with replication and sharding out of scope. It is not an analytics engine: the query language supports equality only, deliberately, because that is what a hash index serves without degrading, so there is no aggregation, sorting, grouping or range querying. And it is not a multi-tenant or internet-facing service: it assumes a trusted local caller, and authentication is not part of this version. Section 3 records these exclusions as the *Will not have* tier.

2 Definitions

Page A fixed-size segment of the database file. Every page is the same size, so the location of page n is a multiplication by set page size.

Collection A named set of records in JSON format; a table without a schema.

Record A single instance of a data object: one JSON document, stored in one slot of one page.

Primary field The field whose value uniquely identifies a record in its collection, chosen at creation and defaulting to `id`.

Canonical text The single text form of a scalar JSON value used for comparison and hashing (Section 6.5). A query value matches a field when the canonical text of the field equals the query value.

Slot One stored entry within a page, described by an entry in the slot directory; its size is whatever its content needs.

Slot directory A table at the start of each page giving every slot's offset, length and status, so entries of differing sizes can be relocated inside a page. Slot numbers have meaning only inside their own page; nothing outside the page refers to them.

Tombstone A slot marked deleted but not yet physically removed, so that a delete costs one flag write.

Compaction Rewriting a page to discard its tombstoned slots and reclaim their space, triggered when tombstones reach 70% of the page's slots.

Bucket One addressable entry of a hash index: page `bucketBase` + b plus any overflow pages chained to it. It holds the records (primary index) or the primary keys (secondary index) whose hashed value maps to b.

Overflow page A page chained to a full bucket page to hold the entries that no longer fit.

Secondary index A hash index over a non-primary field. Its buckets store only the primary keys of matching records, so a record is never duplicated across indexes.

Pager The component that owns the file – reading, writing, caching and allocating pages. Every other component reaches the file through it.

Working set The private copies of the pages one operation has modified. They become visible, and are logged, only when the operation commits; an operation that fails before then changes nothing.

Write-ahead log (WAL) A sequentially written file recording each changed page *before* the database file is relied upon to hold it, so an interrupted operation can be completed or discarded at the next startup.

Commit group The page images of one operation followed by a commit entry in the WAL. A group without its commit entry is ignored by recovery.

Checkpoint Writing every modified cached page to the database file, flushing it to the device, and emptying the WAL.

Fill ratio (load factor) Live entry bytes in an index divided by the bytes available in its bucket pages. It rises as a collection grows, lengthening overflow chains; crossing a threshold triggers an index rebuild.

3 Features and business requirements

Requirements are grouped with the MoSCoW framework. Each has an identifier used elsewhere in this document, a user story, and acceptance criteria stating when it is done. The *Must have* tier defines the first release.

3.1 Must have

M1 – Single-file paged storage
As a developer, I want my whole database to be one file, so that backing it up or shipping it is a file copy. The entire database – records, catalog and indexes – is one file of fixed-size pages. Page size is chosen at creation, recorded in the header, and defaults to 4 KiB; the offset of page n is n times the page size, so locating a page needs no search. *Accepted when:* creating a database produces exactly one file, and a cleanly shut-down database is that one file (while the process runs, its write-ahead log sits beside it); the header states format version, page size and page count; a database created with a non-default page size reopens without the caller restating it.

M2 – Flexible record layout within a page
As a developer, I want to store records of different sizes without declaring them, so that I waste no space on padding. A page holds multiple records of differing sizes, each located by a slot-directory entry giving its offset, length and status, so records can be moved inside the page without their addresses changing. *Accepted when:* there is no padding between records in pages (no space is wasted). Records can be located within a page in $O(1)$ time using the slot directory. (NFR 5.4).

M3 – Tombstone deletion with page compaction
As a developer, I want deletes to be fast and space reclaimed eventually, so that a delete-heavy workload neither stalls nor leaks disk space. Deleting marks the slot and increments the page's tombstone counter rather than rewriting the page. When tombstoned slots reach **70%** of a page's slots, the page is rewritten without them. *Accepted when:* a delete returns without rewriting the

page; a deleted record is never returned by a query; crossing the threshold compacts the page and the space becomes available to later inserts, preserving every live record.

M4 – Hash-based primary index
As a developer, I want a lookup by identifier to be instant however much data I have stored, so that my application stays responsive as it grows. Each collection has a primary field unique to each record. Its value is hashed and reduced modulo the bucket count to select the bucket, and the record is stored in that bucket's page, so a lookup reads one page instead of scanning – $O(1)$ rather than $O(n)$. *Accepted when:* a duplicate or missing primary key is rejected; a lookup by primary key reads a constant number of pages independent of collection size, save for walking an overflow chain.

M5 – Hash-based secondary indexes
As a developer, I want to query by a field other than the identifier without a full scan, so that my common lookups are all fast. Any non-primary field may be indexed: a region of pages is dedicated to the index and the field of every record hashed into its buckets. Unlike the primary index, a bucket holds not record data but an array of the primary keys of matching records, read back through the primary index. *Accepted when:* creating an index backfills it from existing records; a query on an indexed field avoids a full scan; record data is never duplicated; writes keep every index current; a record lacking the indexed field is absent from that index.

M6 – HTTP CRUD API with a query language
As a developer, I want to use the database from any language without installing a driver, so that integration costs me nothing. Create, read, update and delete are exposed over HTTP with JSON bodies. Reads and deletes select records with a query language in one URL parameter, supporting equality combined with `&`, `|` and parentheses (Section 7 is normative). *Accepted when:* all four methods are supported on a collection; a malformed query is rejected with a diagnostic naming the fault position and changes nothing; results are pageable with `limit` and `offset`.

M7 – Multiple collections
As a developer, I want unrelated kinds of data kept apart in one database, so that one file still serves my whole application. A database holds many collections, each with its own primary field and indexes. The pages of one are never read while serving a query against another. *Accepted when:* collections can be created, listed and dropped at runtime; dropping one frees its pages and indexes; two collections may use the same field names without interfering.

M8 – Durable persistent storage
As a developer, I want an acknowledged write to still be there after a restart, so that I can trust the database with real data. Once a write is acknowledged it survives process termination and machine restart. *Accepted when:* data written before a clean shutdown is present after reopening; an acknowledged write is durable per NFR 5.2.

M9 – Crash recovery with a write-ahead log
As a developer, I want an unexpected shutdown to leave my database intact rather than corrupt, so that a power loss is not a data loss. Every change is recorded in a write-ahead log before it is applied to the database file. On startup an unclean shutdown is detected and the log replayed, so an operation is either fully applied or not applied at all. *Accepted when:* the change reaches the log durably before the database file is touched; startup after a kill leaves every acknowledged write present and every unacknowledged one absent; no page is left half-written; a checkpoint empties the log.

M10 – Cross-platform support
As a developer shipping to several platforms, I want the same database on all of them, so that I maintain one storage layer. DBMS runs on Linux, Windows and macOS, and a file written on one is readable on the others. *Accepted when:* the system builds and its tests pass on all three; on-disk integers and text have a platform-independent representation (Section **??**).

3.2 Should have

Valuable and planned, but not required for the first release.

S1 – Multiple concurrent readers
Reads execute concurrently with each other while writes remain serialised. The first release serialises all access (NFR 5.3); this lifts that restriction for readers.

S2 – Result caching
Recent query results are served from memory, invalidated by any write touching the collection concerned.

3.3 Could have

C1 – Data compression
Record payloads are compressed before being written, trading processor time for file size.

C2 – Slotted array optimisation
The in-page slot directory is tuned to reduce per-record overhead and speed up in-page search.

3.4 Will not have

Deliberately excluded, so that an implementer does not design for them and a reviewer does not expect them.

W1 – Sorted or range-based indexing
No ordered index structure, and so no range, prefix or sorting queries. A hash index cannot answer them, and the query language is restricted to equality so that no expressible query outruns the storage engine.

W2 – Fixed schemas and column types
Collections declare no fields and no types beyond the primary field; validating record shape is the application's job.

W3 – Page-level locking
Concurrency is controlled for the database as a whole, not per page.

W4 – Multi-computer networked database
One database, one process, one machine: no replication, clustering, sharding or distributed transactions.

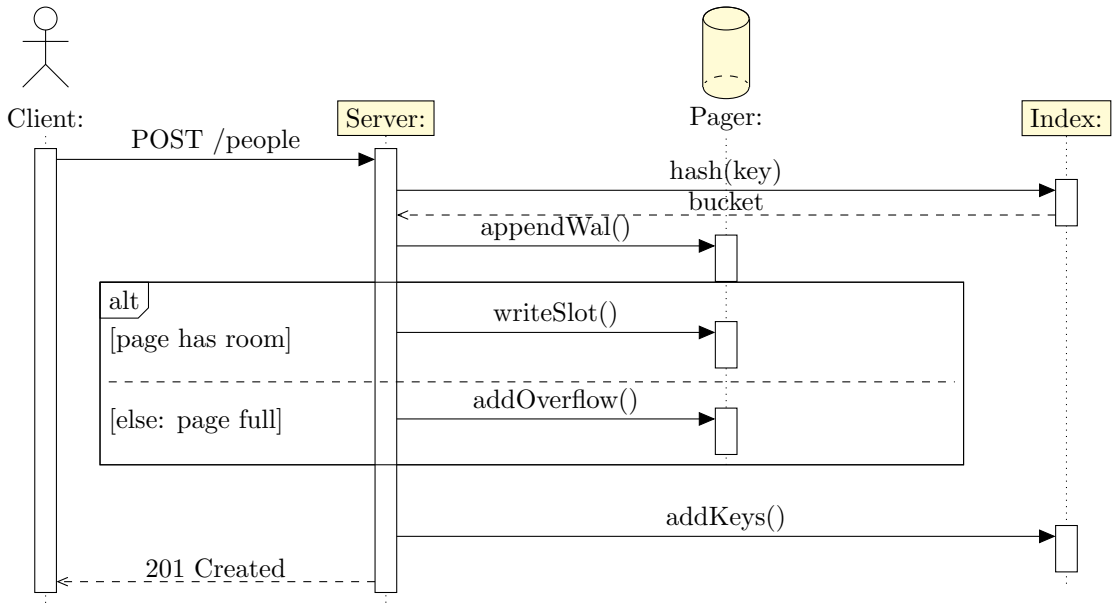
W5 – Authentication and authorisation
DBMS assumes a trusted caller and ships no accounts, passwords or permissions (NFR 5.5).

4 Business flows

Most interactions with DBMS are simple request-response exchanges, fully described by the API reference in Section 7. This section documents the five processes that are *not* trivial, because they involve several components and a decision that changes what happens next. Each is given as a sequence diagram with a written walkthrough.

4.1 Inserting a record

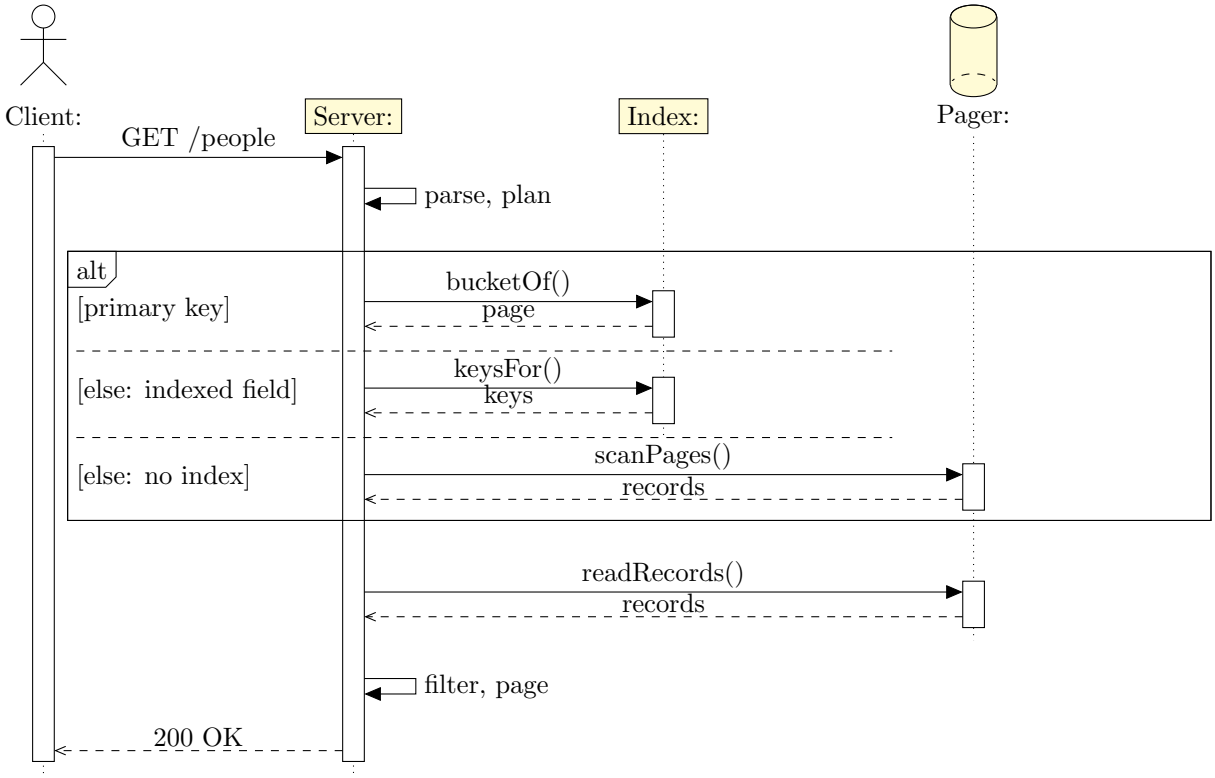
An insert must find the right bucket, find room in it, update every index on the collection, and be recoverable if the machine dies halfway through.



1. The collection is resolved against the catalog, which is held in memory (Section **??**) and costs no disk read; it names the primary field and lists the indexes. An unknown collection ends the flow with `404`.
2. The primary field is validated: missing yields `400`, already present yields `409`. Its value is hashed and reduced modulo the bucket count, giving the page that should hold the record.
3. The intended change is appended to the write-ahead log and flushed to disk. Nothing has touched the database file yet; from here the operation survives a crash.
4. **Decision.** If the target page has room, the record goes into a new slot and the slot directory is extended. If not, an overflow page is allocated, chained to the bucket, and the record written there.
5. Every secondary index is updated: each indexed field present in the record is hashed and the record's primary key appended to the matching bucket.
6. The pages are written, the log marked applied, and `201 Created` returned. Only now is the write acknowledged.

4.2 Querying with an index

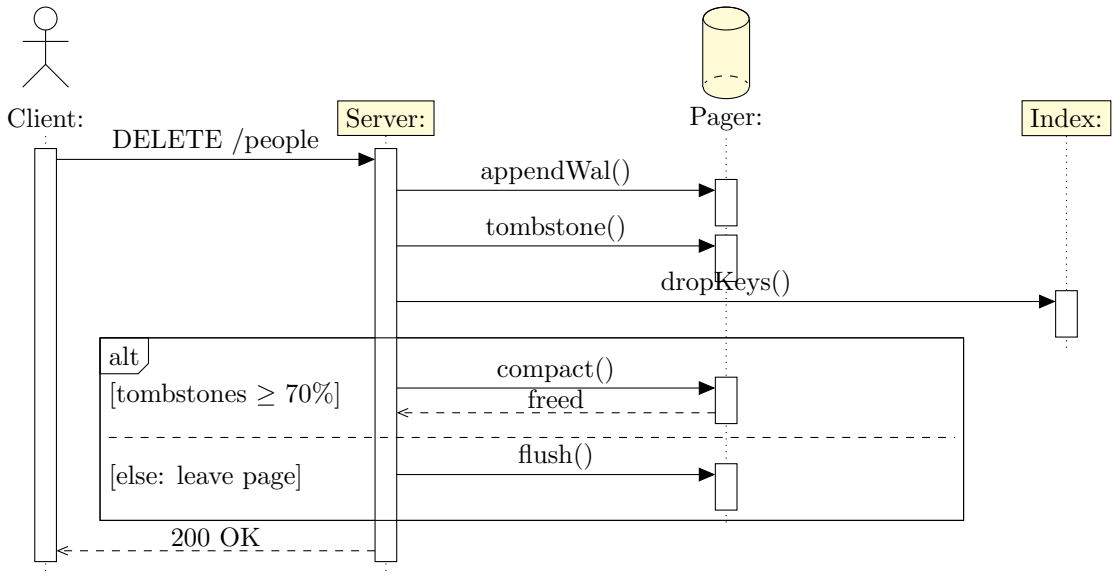
A query's cost is decided entirely by which of three plans the planner can use, so this flow is primarily a decision.



1. The predicate is parsed into a tree of equality clauses joined by and/or. A syntax error ends the flow with `400` and a fault position.
2. **Decision.** The planner picks one of three plans:
   - a clause on the **primary field** resolves to one bucket – one page read, the cheapest case;
   - a clause on an **indexed field** reads the secondary index bucket for a list of primary keys, each then resolved through the primary index at one page read per matched record;
   - with **no usable index**, every page of the collection is read. This is the only $O(n)$ plan and the reason NFR 5.1 states two latency tiers.
3. For an and-expression the most selective indexed clause produces the candidates; for an or-expression each branch is evaluated and the union taken, de-duplicated by primary key. A branch with no usable index forces the whole or-expression to a scan.
4. Candidates are read, tombstoned slots skipped, and the remaining clauses checked in memory – an index narrows the candidates but does not prove the predicate.
5. `limit` and `offset` are applied and the records returned in the envelope of Section 7.

4.3 Deleting records and compacting a page

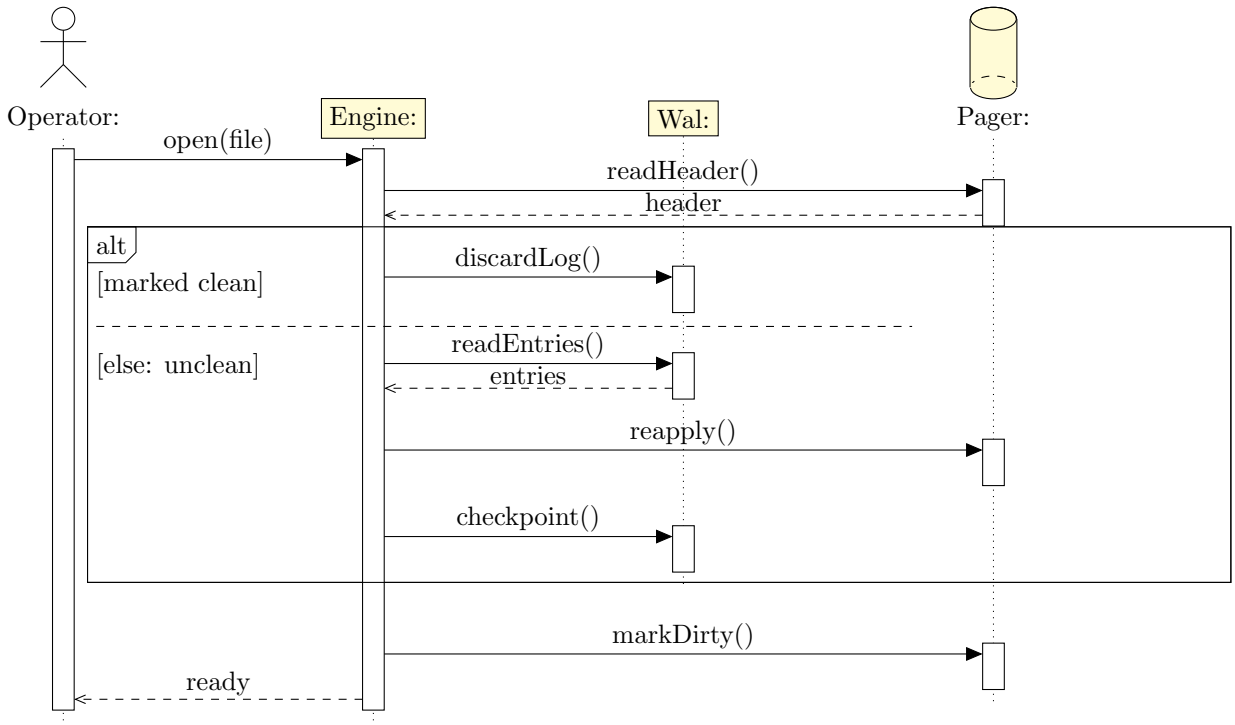
Deletion is cheap by design; the cost is deferred to a compaction that happens only when a page has become mostly garbage.



1. The records are selected exactly as in a query, by the same planner, and the deletion is logged before anything changes.
2. For each match the slot's deleted flag is set and the page's tombstone counter incremented. The record bytes stay in place – this is what makes a delete cheap.
3. The primary key is removed from every secondary index bucket that referenced it, so no index can return a deleted record.
4. **Decision.** The tombstone counter is compared with the slot count. At or above **70%** the page is compacted: live records are copied into a fresh page image, the slot directory rebuilt, the counter reset and the space returned to the free-space map. Below the threshold nothing happens and the space is reclaimed by a later delete.

4.4 Startup and crash recovery

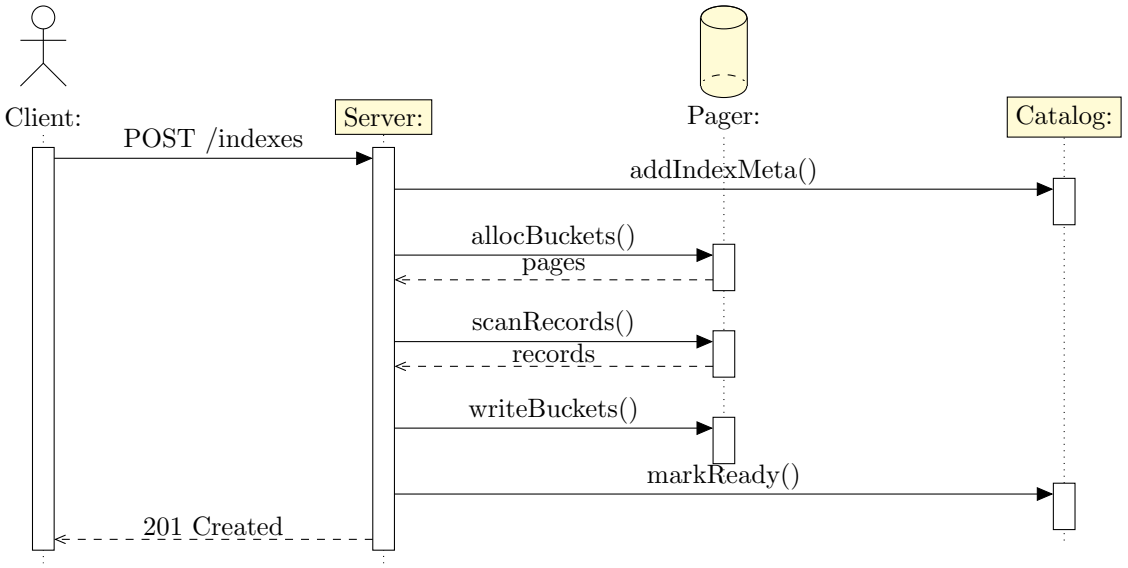
This flow is what makes M9 true. It runs on every startup, and the decision it makes depends on how the database was last closed.



1. The file header is read and its magic value and format version checked; a mismatch aborts the open rather than guessing.
2. **Decision.** The header carries a clean-shutdown flag. **Clean:** the previous run closed properly, so the log is redundant and is discarded. **Unclean:** the process died while running, and the log is read from the last checkpoint forward.
3. Each entry is verified by checksum and replayed in order up to the last complete one; a truncated trailing entry is discarded – exactly the case of a machine that died mid-write. This is why an unacknowledged write never survives and an acknowledged one always does.
4. Replay is idempotent, because an entry records a page's full intended content, so a crash *during recovery* is itself recoverable.
5. A checkpoint flushes the recovered pages and empties the log. The header is marked dirty for the run, and clean again only by an orderly shutdown.

4.5 Creating a secondary index

Adding an index to a collection that already holds records requires a backfill, the one routinely long-running operation in the system.



1. The request names the field and, optionally, a bucket count, defaulting to the collection's record count divided by the target load factor (Section **??**).
2. An index entry is added to the catalog and marked *not ready*, so the planner will not use a half-built index, and a contiguous range of bucket pages is allocated.
3. Every page of the collection is scanned once. For each live record carrying the indexed field, the value is hashed and the record's primary key appended to the matching bucket; records lacking the field are skipped.
4. The index is marked ready and becomes available to the planner. Because the whole operation is logged like any other write, an interrupted backfill leaves no half-built index behind – the index is either absent or complete.

5 Non-functional requirements

These constrain *how well* the system does what Section 3 says, stated so they can be tested rather than argued about. Figures are targets for a mid-range desktop machine with a solid-state drive, on a database of one million records, cache warm unless stated otherwise.

5.1 Performance

Because the engine offers only hash indexing, query cost falls into two sharply different tiers, and the API makes which tier applies predictable rather than surprising. All targets apply to a standard

laptop (4-core CPU, 16 GB RAM, SSD) with a collection of 1,000,000 records of about 1 KB each. They are verified with a test script at the end of the project.

- **Indexed lookup (the $O(1)$ tier).** An equality query on the primary field or an indexed field completes in **under 1 ms** at the 99th percentile, and the pages it reads do not grow with collection size: one page for the primary field, one index page plus one page per matched record for a secondary index.
- **Unindexed lookup (the $O(n)$ tier).** A query with no usable index reads every page of the collection, sustaining at least **200 MB/s** of scanned pages. This tier is documented, not optimised.
- **Writes.** A single-record insert, update or delete completes in **under 5 ms** at the 99th percentile including the log flush that makes it durable, sustaining at least **1,000 durable writes per second**.
- **Overflow degradation.** An indexed lookup walks its bucket's overflow chain, so cost is proportional to chain length. The load-factor threshold of Section **??** holds average chain length near one, and `GET /stats` must warn when a collection exceeds it.
- **Startup.** Opening a cleanly closed database is **under 50 ms** whatever the file size, since only the header and catalog are read; recovery is proportional to log size, not database size.

5.2 Durability and consistency

- Once a write is acknowledged it survives process kill, operating-system crash and power loss. The log entry must therefore reach the physical device before the response is sent, not merely be handed to the operating system.
- An operation is atomic: after recovery it is entirely present or entirely absent. A `DELETE` matching many records is atomic as a whole.
- A half-written page is never visible; recovery either completes the page write from the log or discards it. The file is never left in a state its own format rules forbid: index buckets never reference deleted records, and no live record is unreachable from the primary index.
- Transactions spanning more than one HTTP request are *not* supported – the unit of atomicity is one request.

5.3 Concurrency

- The first release serialises access across the whole database: one operation at a time, others queue. This follows from W3 and makes correctness easy to establish.
- The HTTP server still accepts concurrent connections and queues their requests rather than refusing them, serving at least **64 simultaneous connections**.
- The design must not foreclose S1: all reads go through the pager, so admitting concurrent readers later is a change to one component.
- Exactly one process may open a database for writing, enforced with an advisory file lock; a second attempt fails at once with a clear diagnostic.

5.4 Capacity, limits and resource usage

Property	Limit	On exceeding
Page size	512 B to 64 KiB, default 4 KiB	rejected at creation
Database file size	16 TiB (32-bit page ids)	`507` on insert
Record size	must fit one empty page	`413` on insert
Collections	4,096 per database	`507` on create
Indexes	16 per collection	`507` on create
Field name length	255 bytes	`400`
Page cache	configurable, default 64 MiB	least-recently-used eviction
Request body	256 KiB	`413`

A record larger than a page is rejected rather than split across pages. This is a deliberate simplification: it keeps the pager free of multi-page records, and a 4 KiB page accommodates

realistic JSON documents. Applications needing larger values raise the page size at creation.

Memory is bounded and configurable: the process runs within **128 MiB** resident with the default 64 MiB cache, whatever the database size, because the database is never loaded wholly into memory. Nor does memory grow with result size – results are streamed to the response and `limit` caps what is materialised. Disk space stays proportional to live data, as compaction and tombstone reuse keep a steady-state workload from growing the file without bound.

Portability. Linux, Windows and macOS on 64-bit hardware (M10). No platform-specific facility is used beyond ordinary file operations – open, read, write, flush-to-device, truncate and advisory locking – each isolated behind a single abstraction, so porting touches one place. The format itself is platform-independent: fixed-width little-endian integers, UTF-8 text, nothing depending on host alignment or word size, so a file written on one platform is byte-for-byte usable on another.

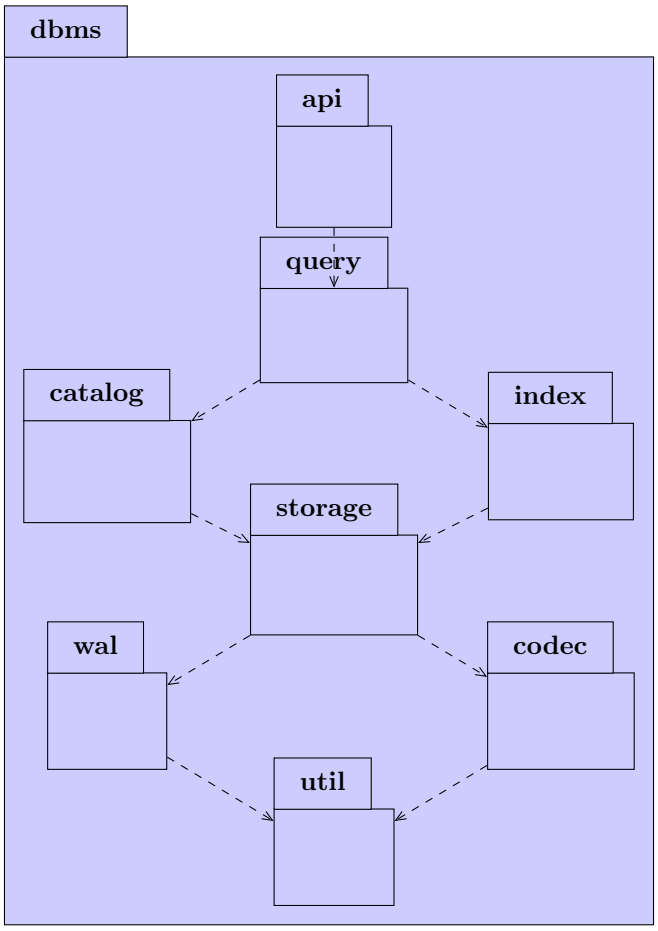
5.5 Reliability, observability and security

- No input may crash the database: malformed JSON, an invalid query, an absurd `limit` or a truncated request yields an error response and the process keeps serving. Errors carry a correct status, a stable machine-readable code and a human-readable message (Section 7), so a client can tell its own fault from the database's.
- Corruption is detected rather than propagated: the header carries a magic value and format version, every log entry a checksum, and a file failing these checks is refused rather than opened optimistically. The format is versioned from the first release.
- Every request is logged with method, collection, **chosen plan**, records matched and duration, the plan included so a developer can tell whether a query used an index. Recovery logs that it occurred, how many entries were replayed and how many discarded as incomplete.
- DBMS has no authentication or authorisation (W5) and assumes a trusted caller, so it binds to loopback by default and the documentation must state plainly that exposing a DBMS port to an untrusted network grants full read and write access. Collection and field names from requests are treated as data, never as file paths.
- The packages of Section 6 depend in one direction only, so each is testable without the layers above it – the storage engine with no HTTP server running. The implementation must offer a hook to abort the process at a chosen point in a write, so recovery from each interruption point is tested rather than hoped for.

6 High-level architecture

6.1 Package structure

DBMS is organised as eight packages. Dependencies run in one direction only – downwards in the diagram below – so that no lower layer knows about a higher one. In particular the storage engine has no knowledge of HTTP or of the query language, which is what makes it testable on its own (NFR 5.1 and the testability requirement).



`api` The network edge: listens, routes a method and path to a handler, parses bodies, renders responses, maps internal errors to HTTP status codes. Contains no storage logic. *Uses* `query`.

`query` The query language: parses the parameter into a tree of equality clauses, decides how to answer it, executes that decision. The planner chooses between the three plans of Section 4, so this is the one place query cost is decided. *Uses* `catalog`, `index`, `storage`.

`catalog` Database metadata: which collections exist, their primary fields and their indexes. Stored in pages of the file, so read like any other data. *Uses* `storage`.

`index` Hashing and buckets: computes a value's hash, maps it to a bucket, walks overflow chains, maintains primary and secondary indexes, and decides when a load factor requires a rebuild. *Uses* `storage`.

`storage` The file: the pager reads, writes, caches and allocates pages; the slot directory places records within a page; the free-space map finds a page with room; the compactor rewrites pages past the tombstone threshold. Every byte to disk passes here. *Uses* `wal`, `codec`.

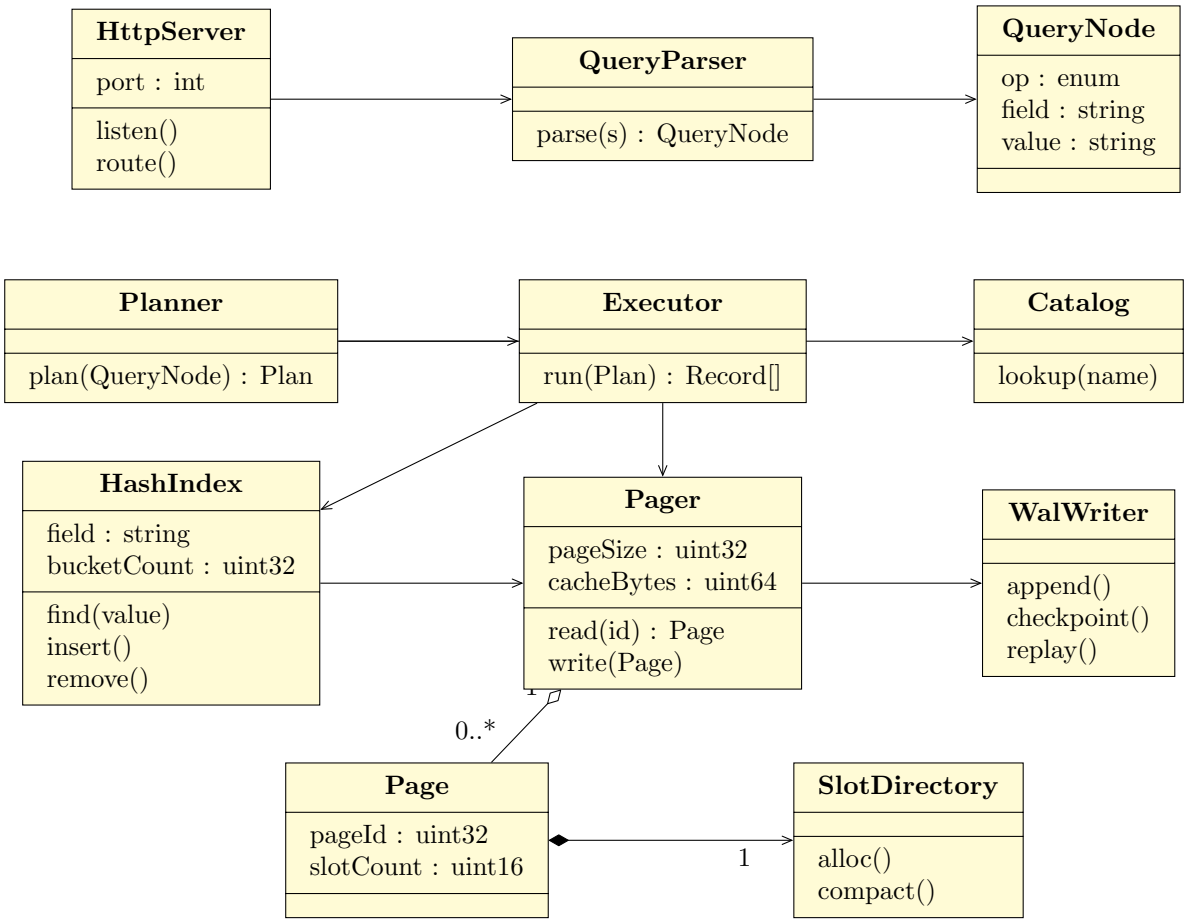
`wal` Durability: appends intended page changes, flushes them to the device, replays on recovery, empties the log at a checkpoint. Deliberately *below* `storage` so no page write can bypass it. *Uses* `util`.

`codec` Byte-level representation: JSON parsing and serialisation, and the fixed-width little-endian encoding of Section **??**. Concentrating it here is what makes the format platform-independent. *Uses* `util`.

`util` Shared primitives with no dependencies: checksums, the hash function, error types, logging, and the file-operations wrapper isolating platform differences.

6.2 Class structure

The diagram below names the principal classes and how they relate. Methods are shown only where they are the component's reason to exist; the full field-level model is Section **??**.



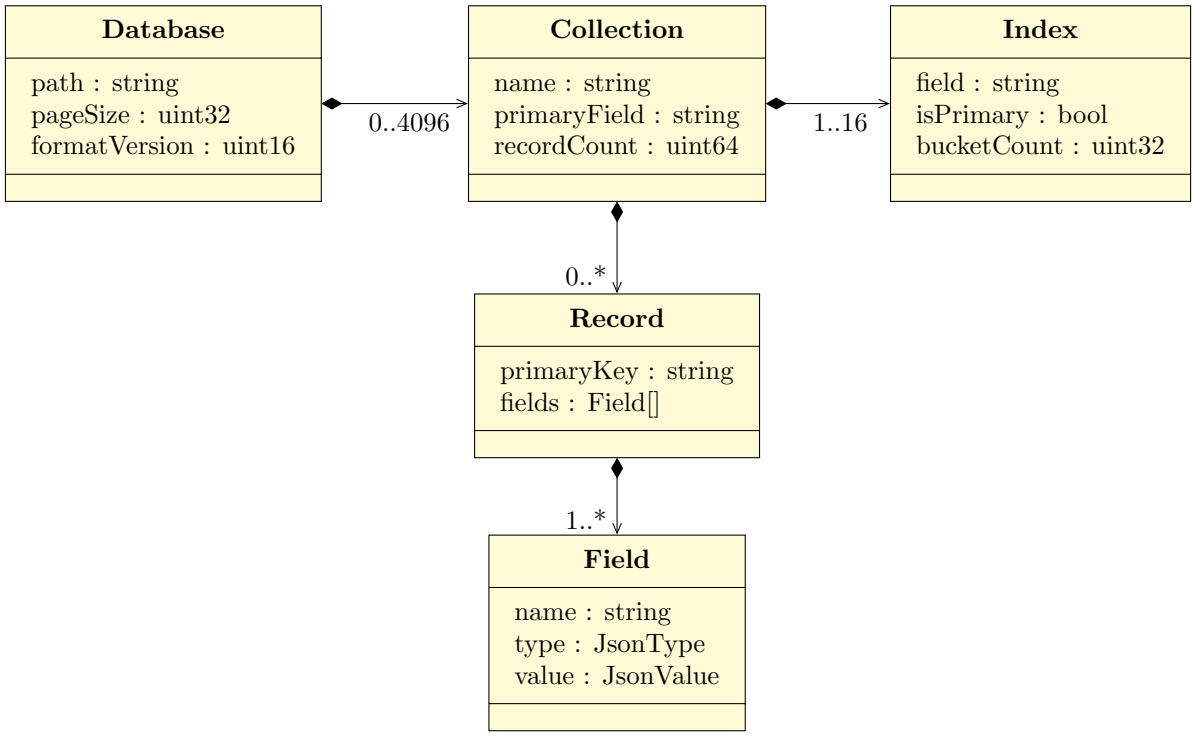
6.3 Mapping requirements onto packages

Each must-have requirement has exactly one package that owns it, so that an implementer knows where work belongs.

Req.	Summary	Owning package
M1	Single-file paged storage	`storage`
M2	Flexible record layout	`storage`
M3	Tombstones and compaction	`storage`
M4	Primary hash index	`index`
M5	Secondary hash indexes	`index`
M6	HTTP API and query language	`api`, `query`
M7	Multiple collections	`catalog`
M8	Durable persistence	`storage`, `wal`
M9	Crash recovery	`wal`
M10	Cross-platform support	`util`, `codec`

The data model has two layers. The *logical* model is what an application sees through the API: collections that hold records, and records that hold fields. The *physical* model is how those things are laid out inside the single database file. This section describes the logical model first, then the file, then how indexes and crash-safe writing work. The exact byte layouts, which an implementer needs, are collected in Section 6.7.

6.4 Logical model



Database One file. It has a page size and a format version, both fixed when the file is created. It can hold up to 4,096 collections.

Collection A named group of records, like a table without a schema. It has a *primary field*, the field whose value identifies each record. A database can hold up to 4,096 collections.

Index A way to find records quickly by one field. Every collection has a primary index, which cannot be removed, and may have secondary indexes. The primary index counts towards the limit of 16 indexes per collection.

Record One JSON document. Because DBMS is schemaless (W2), two records in the same collection may have completely different fields. The only field a record *must* have is the collection's primary field.

Field A name and a JSON value: `string`, `number`, `bool`, `null`, `object` or `array`. Objects and arrays are stored as given, but only a simple (scalar) value can be searched or indexed.

6.5 Keys and how values are compared

A query value is always text, but a field can hold a number or a boolean. To make the two comparable, every simple value has one *canonical text*. A query matches a field when the two texts are exactly equal, with upper and lower case treated as different.

JSON value	Canonical text
string	the string itself
number	the shortest decimal text that reads back as the same number, with no exponent and no trailing `.0`: `34.0` and `3.4e1` both become `34`, and `-0` becomes `0`
`true`, `false`, `null`	the words `true`, `false`, `null`
object, array	none: never matches and is never indexed

In practice this means matching ignores type on purpose: `age=34` finds the number 34 and the string `"34"`, and `active=true` finds the boolean and the string `"true"`. A record that lacks the field never matches. An empty query value matches only the empty string.

Primary keys. A record's *primary key* is the canonical text of its primary field. That field must hold a string of 1 to 255 bytes or a whole number no larger than 2^{53}; anything else is `400 INVALID_KEY`, and a missing field is `400 MISSING_KEY`. Keys are compared as canonical text, so the number 42 and the string `"42"` are the same key, and a second insert of either is a duplicate.

How a record is stored. The JSON is kept as sent, without extra whitespace, with fields in their original order and numbers as written. A record with a repeated field name, or nested more than 32 levels deep, is rejected (`400 INVALID_JSON`). JSON numbers are interpreted using double-precision numeric values when their canonical text is produced. The original JSON representation is still stored unchanged. As a result, whole numbers above 2^{53} are preserved in the stored JSON, but their numeric values may not be distinguishable exactly during comparison.

6.6 Physical model

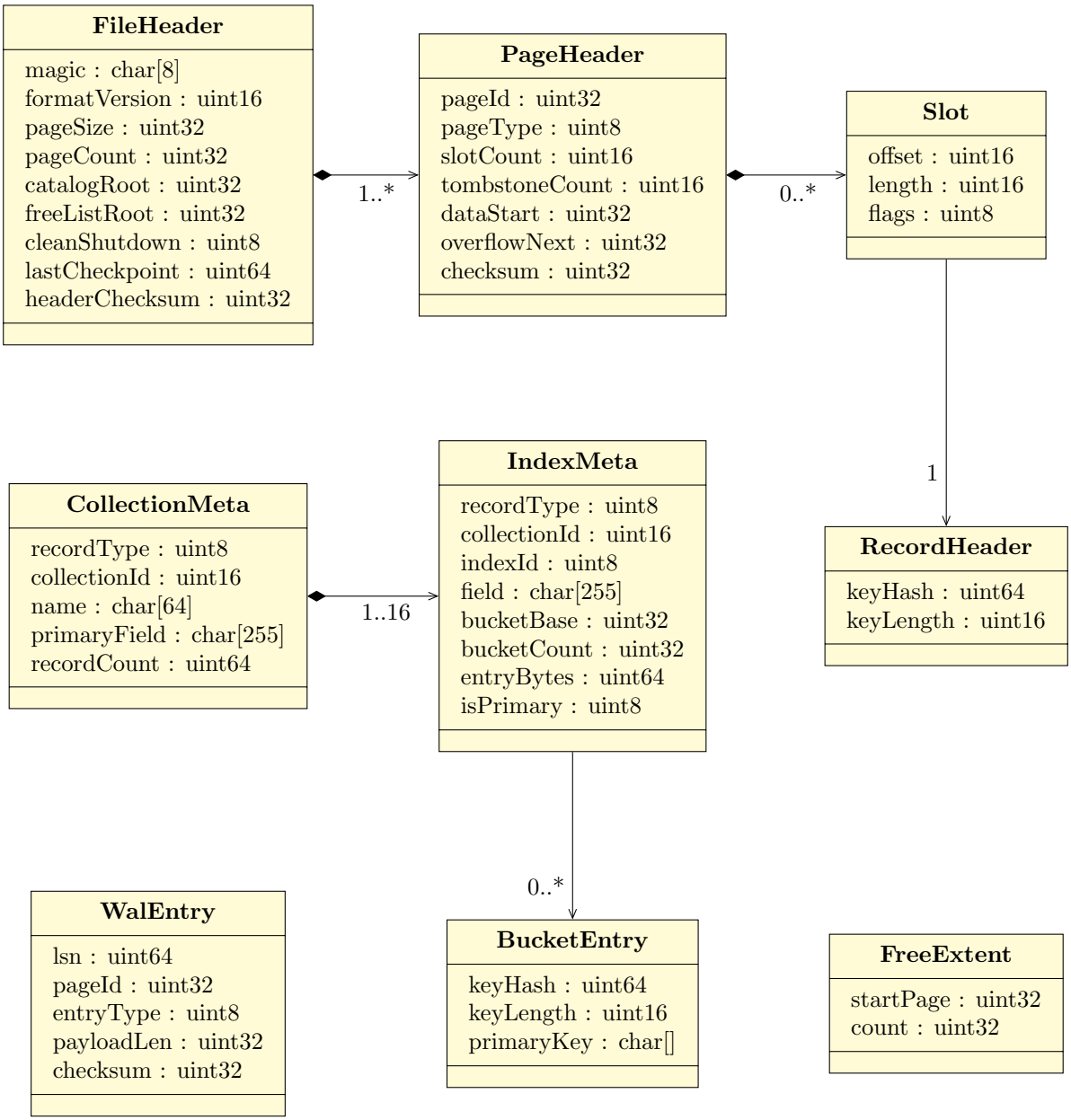
The file is a list of pages. Every page has the same size, set when the database is created (512 B to 64 KiB, default 4 KiB). Page 0 is the file header. Every other page has a small header and a *type* that says what it holds:

Type	Name	What it holds
1	Catalog	the list of collections and indexes
2	Primary bucket	the records themselves, grouped by the hash of their key
3	Secondary bucket	primary keys of records, grouped by the hash of an indexed value
4	Overflow	extra entries from a bucket page that has filled up
5	Free list	a list of pages that are free for reuse

A collection's records live in the pages of its primary index and their overflow pages, and nowhere else. Inside a page, entries of any size sit side by side: a small directory of *slots* at the start of the page says where each entry is, and the entries themselves fill the page from the far end. Deleting an entry only marks its slot (a *tombstone*); the space is reclaimed later when the page is tidied (*compaction*).

A record is stored as one complete entry and is never split across pages. Therefore, its maximum size depends on the database's page size and the space required for its page slot and record header. A record that cannot fit in an otherwise empty page is rejected with `413 RECORD_TOO_LARGE`. This limit is separate from the HTTP request-body limit of 256 KiB.

The structures are summarised below as fields and types; the byte positions follow in Section 6.7.



A page header is followed by its slot directory, and each slot points to one entry in the same page. Which kind of entry depends on the page type: a record (with its `RecordHeader`), a bucket entry, a catalog record or a free extent. `CollectionMeta` and `IndexMeta` are the two kinds of catalog record, linked by `collectionId`. `WalEntry` is different: it lives in the log file, not in the database file.

6.7 Byte layouts

These are the exact formats. Three rules apply everywhere: numbers are stored little-endian at the widths shown; text is UTF-8; and a `char[N]` field is always exactly N bytes, padded with zero bytes on the right. Together they make a file usable on any platform (M10). Offsets count bytes from the start of the structure. Every checksum is **CRC-32** (the common zlib/Ethernet variant: polynomial `0xEDB88320`, start and end value `0xFFFFFFFF`).

Off.	Field	Type	Meaning
File header – page 0			
0	`magic`	char[8]	the text `"PSPDB000"`; any other file is refused
8	`formatVersion`	uint16	format version, currently 1
10	`pageSize`	uint32	bytes per page: a power of two from 512 to 65536
14	`pageCount`	uint32	pages that belong to the database; any later pages in the file are ignored
18	`catalogRoot`	uint32	first catalog page
22	`freeListRoot`	uint32	first free-list page, 0 if none
26	`cleanShutdown`	uint8	1 if the last close was orderly; decides whether recovery runs
27	`lastCheckpoint`	uint64	log number of the last checkpoint
35	`headerChecksum`	uint32	CRC-32 of bytes 0–34
39	`reserved`	byte[]	zeros to the end of the page
Page header – every other page (21 bytes)			
0	`pageId`	uint32	the page's own number, to catch misplaced pages
4	`pageType`	uint8	1 catalog, 2 primary bucket, 3 secondary bucket, 4 overflow, 5 free list
5	`slotCount`	uint16	slots in the directory, live and deleted
7	`tombstoneCount`	uint16	deleted slots; the page is compacted when `tombstoneCount` reaches 70% of `slotCount`
9	`dataStart`	uint32	where entry data begins (the lowest used byte); the page size if the page has no entries
13	`overflowNext`	uint32	next page in the chain; for bucket and overflow pages it links to the next overflow page; for free-list pages it links to the next free-list page. Zero means there is no next page.
17	`checksum`	uint32	CRC-32 of the whole page, with this field counted as zero

Inside a page. The slot directory starts at byte 21 and grows towards the end of the page; entry data starts at the end of the page and grows towards the front. `dataStart` marks the boundary. The free space is `dataStart` $-$ $(21 + 5 \times$ `slotCount`$)$, and an entry of n bytes fits when $n + 5$ is no more than that – the test used in the insert flow (Section 4). Each slot is five bytes: `offset` (uint16, where the entry starts), `length` (uint16) and `flags` (uint8, bit 0 set means deleted). Sixteen bits are enough even for a 64 KiB page, because an entry is always smaller than a page. A slot's number is only its position in the page's directory; it can change when the page is compacted, and nothing outside the page refers to it (M2).

An overflow page that compaction leaves with no live entry is unlinked and freed. A bucket page is never freed, except together with its whole index.

Off.	Field	Type	Meaning
Record (primary bucket and overflow pages)			
0	`keyHash`	uint64	hash of the primary key, so non-matching records can be skipped quickly
8	`keyLength`	uint16	length of the key in bytes
10	`key`	char[]	the primary key (canonical text), `keyLength` bytes
–	`payload`	byte[]	the JSON document, to the end of the entry
Bucket entry (secondary bucket and overflow pages)			
0	`keyHash`	uint64	hash of the *primary key* of the record it points to
8	`keyLength`	uint16	length of that key
10	`primaryKey`	char[]	primary key of the record it points to
Catalog record (catalog pages)			
0	`recordType`	uint8	1 = collection, 2 = index
collection (330 bytes): 1 `collectionId` uint16; 3 `name` char[64]; 67 `primaryField` char[255]; 322 `recordCount` uint64			
index (276 bytes): 1 `collectionId` uint16; 3 `indexId` uint8; 4 `field` char[255]; 259 `bucketBase` uint32; 263 `bucketCount` uint32; 267 `entryBytes` uint64; 275 `isPrimary` uint8			
Free-list page (type 5)			
`slotCount` extents of 8 bytes starting at byte 21: `startPage` uint32, `count` uint32			
Log file header			
0	`magic`	char[8]	the text `"PSPWAL00"`
8	`pageSize`	uint32	must equal the database's page size
Log entry (repeated after the log header)			
0	`lsn`	uint64	log number, always increasing
8	`pageId`	uint32	the page this entry is for; 0 for a commit entry
12	`entryType`	uint8	1 page image, 2 commit
13	`payloadLen`	uint32	length of the payload
17	`checksum`	uint32	CRC-32 of bytes 0–16 and the payload
21	`payload`	byte[]	type 1: the full page; type 2: a uint32 count of the page images in this commit group

6.8 How indexes find records

Hashing. An index turns a value into a number (a *hash*) and uses it to pick a *bucket*, so a lookup goes straight to the right page instead of searching. The hash function is **FNV-1a, 64-bit**, applied to the canonical text of the value: start from `0xcbf29ce484222325`; for each byte, XOR it into the hash, then multiply by `0x100000001b3`, keeping only the low 64 bits. The bucket number is the hash modulo the index's `bucketCount`. Bucket b is page `bucketBase` $+ b$, so no lookup table is needed.

- The **primary index** hashes the primary key. Its bucket pages hold the records themselves, so finding a record by key starts with a direct bucket lookup. If the bucket has overflow pages, additional pages may need to be read.
- A **secondary index** hashes the value of the indexed field. Its buckets hold only the primary keys of matching records. Several different values can share a bucket, so the index gives *candidates*: each is fetched through the primary index and checked against the query.
- When a bucket page is full, extra entries go into overflow pages chained to it.

Worked example. Insert {`"id":"42","name":"John","surname":"Doe","age":34`} into a collection whose primary index has 64 buckets. The JSON is 50 bytes and the key `42` is 2 bytes, so the entry is $10 + 2 + 50 = 62$ bytes, plus 5 for its slot. The FNV-1a hash of `42` is `0x07EE7E07B4B19223`, which modulo 64 is 35, so the record goes into bucket page `bucketBase` $+ 35$. If `surname` is indexed,

the hash of `Doe` is `0xE0B75919922BCEE3`. Its bucket is determined by taking that hash modulo the secondary index's own `bucketCount`, and a small entry holding the key `42` goes there. (Use these two hashes as test values.)

Keeping indexes fast. Buckets fill up as a collection grows, and long overflow chains slow lookups down. DBMS measures this with the *fill ratio*: the bytes of live entries in an index divided by the room in its bucket pages, that is `entryBytes` / (`bucketCount` × (`pageSize` − 21)). The server keeps `entryBytes` up to date in the catalog.

- The aim is a fill ratio of **0.5**. When a write pushes an index above **0.8**, the server rebuilds it after answering the request (Section **??**), into a larger set of buckets sized to bring the ratio back to 0.5 or below, and at least twice as many. Changing the bucket count this way is the only way it ever changes.
- A new collection's primary index starts with **64** buckets unless the request names a number (1 to 16,777,216).
- A secondary index created over existing records starts with the smallest power of two that is at least 32 × `recordCount` / (0.5 × (`pageSize` − 21)), and never fewer than 16. The 32 is an assumed average entry size; a poor guess is corrected by the rebuild rule.
- Because every bucket is at least one page, an index costs about twice its data. A million 100-byte records settle at 65,536 buckets: 256 MiB at the default page size. Every index occupies at least its starting pages (64 for a new collection's primary index, and at least 16 for a secondary index).

6.9 Writing safely: the log, checkpoints and free space

The idea. Before any change is relied on, DBMS writes it to a separate *log file*, named like the database with `.wal` added. If the program stops half-way, the next start uses the log to finish or discard the change, so the database is never left half-written. The log is emptied regularly and deleted on a clean shutdown, so a cleanly closed database is the single file that M1 requires.

Commit groups. Each operation (Section 4) collects its changes in a private *working set* of page copies. To commit, the server appends the complete image of every changed page to the log, then one commit entry that records how many images the group has, and flushes the log to the device. Only then does it answer the client. The changed pages always include the catalog page holding the updated counters, and page 0 if the file grew, so a single-record write logs typically two to four pages. Logging whole pages means applying an entry twice has the same effect as once, so a crash *during* recovery is itself recoverable (M9).

Checkpoints. The database file is updated only at a *checkpoint*, or when a modified page is pushed out of the cache. That is safe because every modified page is already in the log, and the log is emptied only after a checkpoint has flushed all modified pages to the device. A page torn by a crash is therefore repaired from the log. When the checkpoints run is described in Section 4.

Recovery. After an unclean stop, the server reads the log from the start and applies each commit group in order. A group counts only if its commit entry is present and its image count matches; reading stops at the first entry that is cut short or fails its checksum, and anything after the last good commit is thrown away, because that operation was never acknowledged. Log numbers continue from `lastCheckpoint`.

Free space. `freeListRoot` starts a chain of free-list pages, each listing *extents*: runs of consecutive free pages. Freeing a million-page index therefore writes one extent, not a million entries. Single pages are taken from the first extent that has one; if none is free, the file grows. A run of bucket pages is *always* taken by growing the file, so a fragmented free list can never stop an index from being created. Freed pages are reused for overflow pages and later allocations, and free pages at the very end of the file are cut off at a checkpoint. Pages beyond `pageCount` belong to nothing and are ignored, which is how an index built before a crash disappears cleanly.

6.10 The catalog

The catalog is the list of collections and indexes. It lives in catalog pages chained from `catalogRoot`. Each collection and each index is its own small record in its own slot (at most 330 bytes, so it fits even a 512-byte page), and an index record names its collection by `collectionId`. Names are fixed-width and padded with zeros: a collection name uses 64 bytes, a primary field name or indexed field path 255 bytes, with no terminator. Longer names are rejected (`400 INVALID_NAME`), never cut short.

The catalog is read once at startup and kept in memory, so a request can find a collection without touching the disk. Every change to it goes through the log like any other page. This includes `recordCount` and `entryBytes`, which are updated in the same commit group as the write that changed them, so they can never disagree with the data.

7 Interface: HTTP API and query language

The HTTP API is the *only* way to interact with DBMS – there is no shell, no desktop client and no administration console – so this section is the complete interface definition and is normative. The server listens on loopback, port **7070** by default (NFR 5.5); every body is JSON with content type `application/json`, and a request with another content type and a body is refused with `415`. A request body larger than 256 KiB is refused with `413`.

Names. A collection name is 1 to 64 characters from `A--Z`, `a--z`, `0--9`, `_` and `-`, and may not be `collections`, `health` or `stats`, which are fixed paths; otherwise `400 INVALID_NAME`.

7.1 Query language

A query selects records. It is passed as a **single URL parameter** named `query`, and it supports equality comparison only (W1), combined with and, or and parentheses:

```
query   = orExpr
orExpr  = andExpr ( "|" andExpr )*
andExpr = term ( "&" term )*
term    = "(" orExpr ")" | comparison
comparison = field "=" value
field   = 1*( ALPHA / DIGIT / "_" / "." ) ; at most 255 bytes
value   = *( plain / escape )
plain   = any character except  &  |  (  )  =  \
escape  = "\" ( "&" / "|" / "(" / ")" / "=" / "\" )
```

`&` binds tighter than `|`, so `a=1&b=2|c=3` means `(a=1&b=2)|c=3`; parentheses override this.

Encoding. The server first URL-decodes the `query` parameter, then parses the resulting predicate according to the grammar above. Consequently, the logical AND operator `&` must be percent-encoded as `%26` in the URL. To include a literal ampersand in a value, write `\&` in the predicate and URL-encode the backslash. The same escaping rule applies to the other reserved characters listed in the grammar. A literal percent sign must be encoded as `%25`, a plus sign as `%2B`, and a hash sign as `%23`. Values are compared after URL decoding and query-language unescaping. Comparison is exact and case-sensitive.

Nested fields. A dot addresses a field of a nested object – `address.city=Vilnius` matches a record whose `address` object has that `city` – and a dotted path may be indexed like any other scalar field.

Examples.

```
GET /people                       # all records (no query parameter)
GET /people?query=id=42           # by primary key - one page read
GET /people?query=(name=John%26surname=Doe)|(name=Bob%26surname=Smith)
GET /people?query=address.city=Vilnius&limit=50&offset=100
```

7.2 Result paging

`limit` and `offset` are separate parameters, outside the predicate, and apply to reads only.

Parameter	Default	Meaning
`query`	none (match all)	the predicate above
`limit`	100	maximum records to return, 1 to 10,000
`offset`	0	records to skip before returning any

Record order is unspecified and must not be relied upon: DBMS has no ordered index (W1), so records emerge in whatever order the access path visits them. Order is stable for an unchanged collection, which makes `offset` usable for paging a quiescent one.

7.3 Endpoints

Method	Path	Purpose
Records		
`POST`	`/{collection}`	insert one record from the JSON body
`GET`	`/{collection}`	return records matching `query`; paged
`PUT`	`/{collection}`	replace the record whose primary key the body carries
`DELETE`	`/{collection}`	delete records matching `query`
Collections		
`POST`	`/collections`	create a collection
`GET`	`/collections`	list collections with record counts
`DELETE`	`/collections/{name}`	drop a collection, its records and its indexes
Indexes		
`POST`	`/collections/{n}/indexes`	create a secondary index and backfill it
`GET`	`/collections/{n}/indexes`	list indexes with load factors
`DELETE`	`/collections/{n}/indexes/{f}`	drop a secondary index

Insert. The body is the record. It must contain the collection's primary field.

```
POST /people   { "id": "42", "name": "John", "surname": "Doe", "age": 34 }
201 Created    { "inserted": 1, "primaryKey": "42" }
```

Read. Results arrive in an envelope that separates the page from the total, so a client can tell whether more records remain.

```
GET /people?query=(name=John)&limit=2    ->  200 OK
{ "total": 1284, "limit": 2, "offset": 0,
  "plan": "secondary-index:name",
  "records": [ { "id": "42", "name": "John", "surname": "Doe", "age": 34 },
               { "id": "77", "name": "John", "surname": "Smith" } ] }
```

`total` is the number of records matching the predicate, not the number returned. `plan` names the access path chosen – `primary-index`, `secondary-index:{field}` or `full-scan` – so a developer can see whether a query used an index without reading the server log (NFR 5.1).

Update and delete. `PUT` replaces a whole record, located by the primary key in its body; partial update is not supported. A `DELETE` without a `query` parameter is rejected with `400` rather than emptying the collection, because deleting everything by accident is too easy – emptying a collection is done by dropping it.

```
PUT /people   { "id": "42", "name": "John", "age": 35 }   ->  200  { "updated": 1 }
DELETE /people?query=id=42                                ->  200  { "deleted": 1 }
```

Collections and indexes. `primaryField` defaults to `id` and `bucketCount` to 64 (Section **??**); creating a collection that already exists yields `409`. An index request returns once the backfill of Section 4 is complete, so it may take a while on a large collection, and the primary index cannot be dropped.

```
POST /collections   { "name": "people", "primaryField": "id", "bucketCount": 1024 }
201 Created         { "created": "people", "primaryField": "id", "bucketCount": 1024 }
```

`primaryField` defaults to `id` and `bucketCount` to 64 (Section **??**). Creating a collection that already exists yields `409`.

```
POST /collections/people/indexes   { "field": "surname" }
201 Created   { "field": "surname", "bucketCount": 2048, "backfilled": 1284 }
```

The request returns once the backfill of Section 4 is complete, so it may take a while on a large collection. The primary index cannot be dropped.

Operations.

```
GET /health -> 200  { "status": "ok", "uptimeSeconds": 8122 }
GET /stats  -> 200
{ "pageSize": 4096, "pageCount": 25600, "cacheHitRate": 0.97,
  "checkpointPending": false,
  "collections": [ { "name": "people", "records": 1284, "pages": 412,
      "tombstones": 37, "indexes": [
        { "field": "id", "primary": true, "loadFactor": 1.25 },
        { "field": "surname", "loadFactor": 2.6,
          "warning": "above threshold, rebuild advised" } ] } ] }
```

7.4 Status codes and error codes

Every error response carries a stable `code` for programmatic handling and a `message` for a human, e.g. {`"code":"QUERY_SYNTAX", "message": "unexpected ')' at position 18"`}.

Status	Code	Raised when
200	–	read, update or delete succeeded
201	–	insert, collection or index created
400	`QUERY_SYNTAX`	the predicate does not parse
400	`INVALID_JSON`	the body is not well-formed JSON
400	`MISSING_KEY`	the record has no primary field
400	`MISSING_QUERY`	a `DELETE` arrived with no predicate
400	`UNINDEXABLE_FIELD`	an index was requested on an object or array
404	`NO_COLLECTION`	the named collection does not exist
404	`NO_RECORD`	a `PUT` named a primary key that is absent
409	`DUPLICATE_KEY`	the primary key already exists
409	`ALREADY_EXISTS`	the collection or index already exists
413	`RECORD_TOO_LARGE`	the record exceeds one page, or the body the limit
500	`INTERNAL`	an unexpected fault; nothing was changed
503	`RECOVERING`	the database is replaying its log and not yet serving
507	`LIMIT_REACHED`	a capacity limit of NFR 5.4 was reached

A failed request changes nothing: no partial write survives a rejection, which is NFR 5.2 expressed at the interface.